

Patrón SAGA — Creación de Procesos de Selección de Personal CAS

Autor: Percy Giancarlo Guillen Alcantara
Stack: Java · Spring Boot · RabbitMQ · MySQL · MongoDB · CQRS

Tabla de Contenidos

- 1. [Visión General](#)
- 2. [Arquitectura del Sistema](#)
- 3. [Microservicios](#)
- 4. [Patrón CQRS aplicado](#)
- 5. [Flujo de la Transacción Distribuida \(11 Pasos\)](#)
- 6. [Colas RabbitMQ](#)
- 7. [Modelos de Datos](#)
- 8. [Mecanismo de Compensación](#)
- 9. [Request / Response de Ejemplo](#)
- 10. [Estados de la SAGA](#)
- 11. [Consideraciones Técnicas](#)

1. Visión General

El proyecto implementa el **Patrón SAGA con orquestación** para gestionar la creación de un Proceso de Selección de Personal CAS (Contratación Administrativa de Servicios). Al tratarse de una operación que involucra múltiples bases de datos y servicios independientes, se requiere garantizar la consistencia eventual sin utilizar transacciones distribuidas clásicas (2PC).

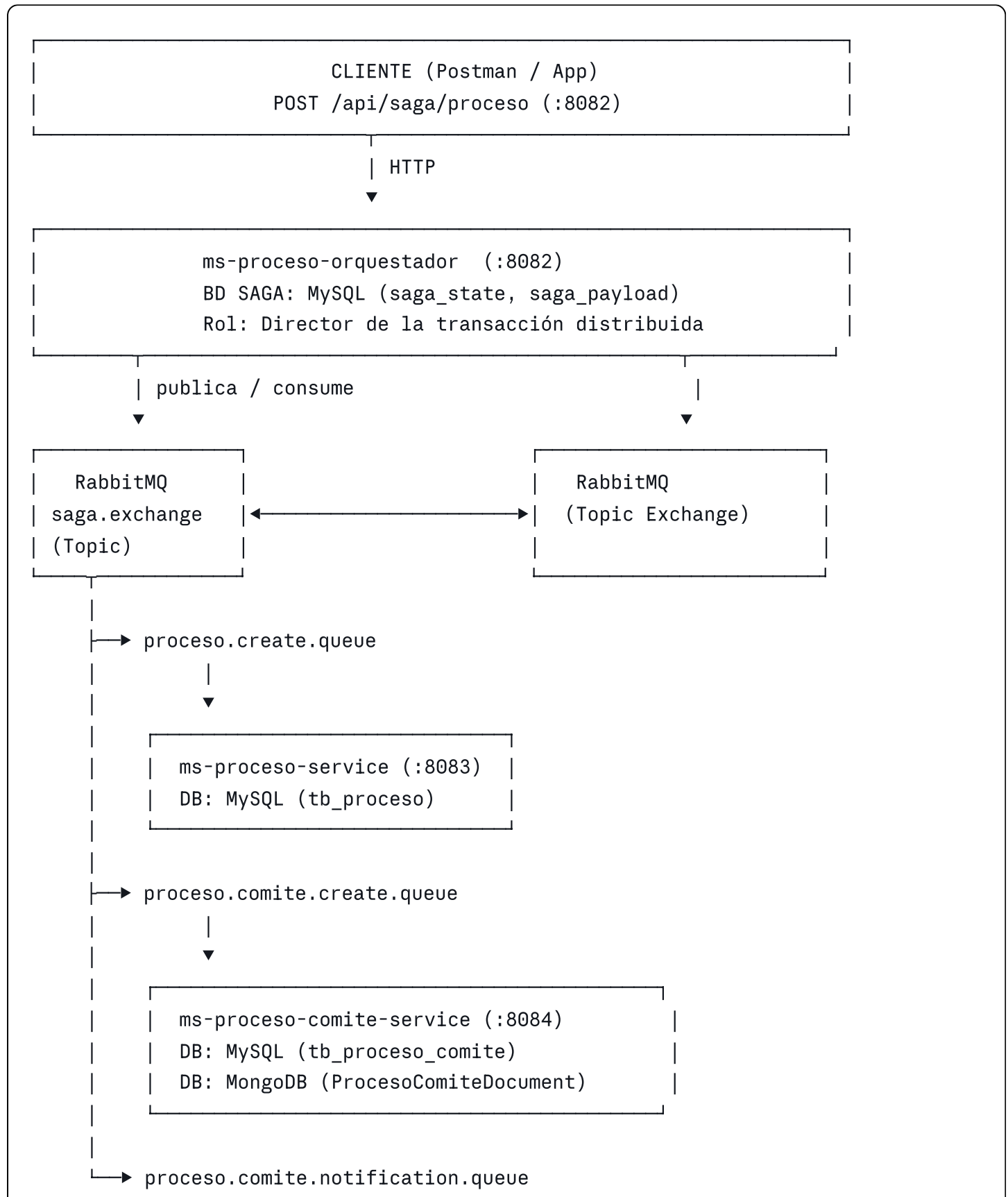
¿Por qué SAGA?

Problema	Solución con SAGA
Una sola transacción no puede abarcar múltiples microservicios	Cada servicio ejecuta su transacción local y comunica el resultado
Si falla un paso intermedio, los datos quedan inconsistentes	Se ejecutan transacciones compensatorias para revertir pasos anteriores
Acoplamiento fuerte entre servicios	Comunicación asíncrona mediante RabbitMQ (desacoplamiento)

Tipo de SAGA implementada

Se utiliza el tipo **Orquestación** (no coreografía), donde un orquestador central — `ms-proceso-orquestador` — dirige el flujo completo, enviando comandos a cada microservicio y escuchando sus respuestas.

2. Arquitectura del Sistema





```
ms-proceso-comite-notification-service  
(:8085)  
Envío de correo electrónico (SMTP)
```

3. Microservicios

3.1 ms-proceso-orquestador (Puerto 8082)

- **Responsabilidad:** Coordinar la SAGA completa. Recibe la petición inicial, publica comandos y reacciona a eventos de éxito o fallo.
- **Base de datos:** MySQL — tablas de control del estado SAGA y payload del comité.
- **Tecnologías:** Spring Boot, Spring AMQP, Spring Data JPA.

3.2 ms-proceso-service (Puerto 8083)

- **Responsabilidad:** Registrar el Proceso CAS en la base de datos relacional.
- **Base de datos:** MySQL — `tb_proceso`.
- **Tecnologías:** Spring Boot, Spring AMQP, Spring Data JPA.
- **Patrón CQRS:** Separación entre `CreateProcesoCommand` (escritura) y queries de lectura.

3.3 ms-proceso-comite-service (Puerto 8084)

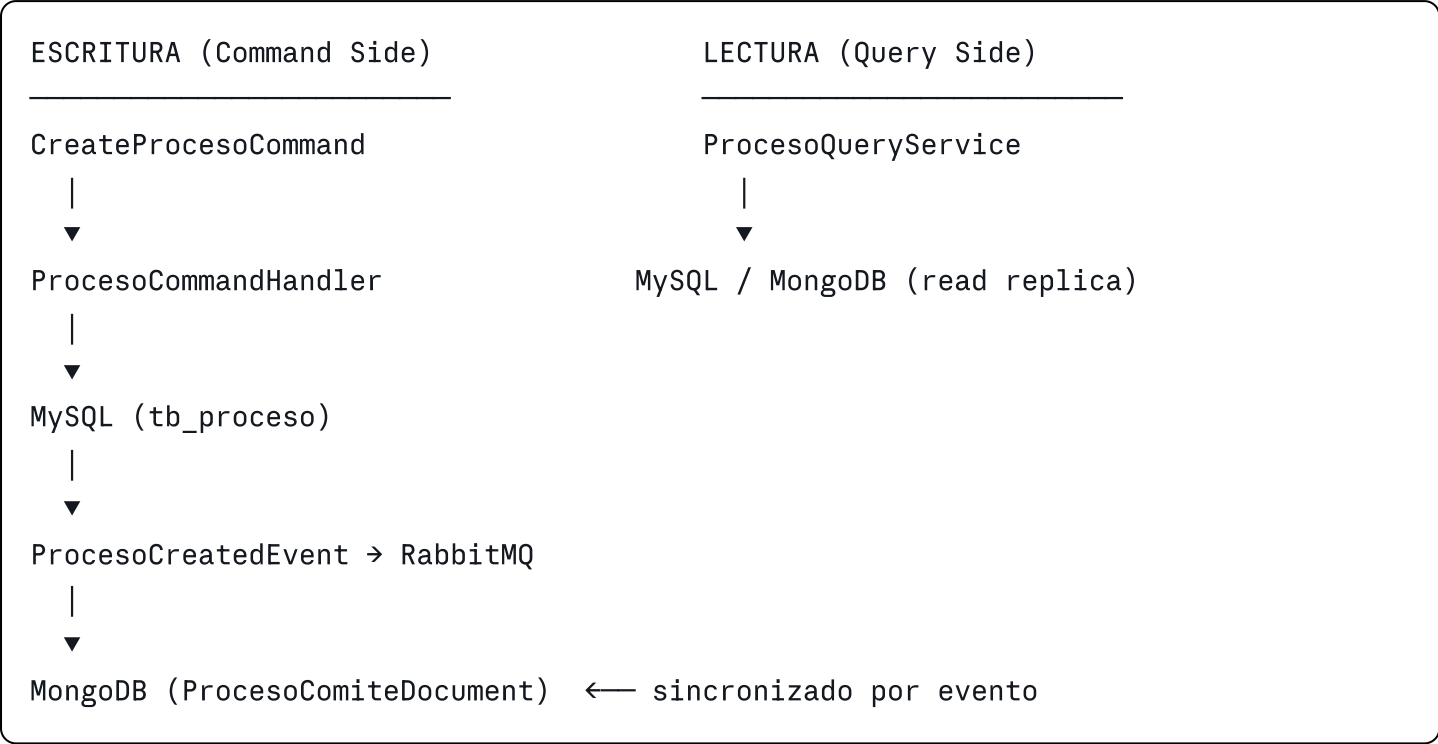
- **Responsabilidad:** Registrar los miembros del Comité de Evaluación con persistencia híbrida.
- **Bases de datos:** MySQL (`tb_proceso_comite`) + MongoDB (`ProcesoComiteDocument`).
- **Tecnologías:** Spring Boot, Spring AMQP, Spring Data JPA, Spring Data MongoDB.
- **Persistencia híbrida:** MySQL para relaciones estructuradas; MongoDB para documentos de respaldo y consultas rápidas (CQRS — lado lectura).

3.4 ms-proceso-comite-notification-service (Puerto 8085)

- **Responsabilidad:** Notificar por correo electrónico a los integrantes del comité.
- **Tecnologías:** Spring Boot, Spring AMQP, Spring Mail (JavaMailSender).

4. Patrón CQRS aplicado

CQRS (Command Query Responsibility Segregation) separa las operaciones de escritura (Commands) de las de lectura (Queries) en modelos distintos.



- El **lado comando** escribe en MySQL transaccional.
- El **lado consulta** lee de MongoDB (alta velocidad, sin bloqueos).
- La sincronización ocurre de forma asíncrona vía eventos en RabbitMQ.

5. Flujo de la Transacción Distribuida (11 Pasos)

Paso 1 — Inicialización de la Solicitud

El cliente emite una petición `HTTP POST` al orquestador:

```
POST http://localhost:8082/api/saga/proceso
```

Acciones del orquestador:

1. Genera un `sagaId` único (UUID).
2. Persiste el payload del comité en la BD SAGA.
3. Establece estado inicial: `STARTED`.
4. Publica mensaje en RabbitMQ → cola `proceso.create.queue`.

Paso 2 — Encolado del Evento de Creación

El orquestador encola el comando de creación y registra los integrantes del comité en su BD de control para uso posterior.

Colas involucradas:

- Publica en: `proceso.create.queue`
-

Paso 3 — Registro del Proceso en MySQL

`ms-proceso-service` escucha `proceso.create.queue`:

1. Extrae el `CreateProcesoCommand` del mensaje.
 2. Persiste el registro en MySQL (`tb_proceso`).
-

Paso 4 — Confirmación o Fallo del Registro Base

Tras la persistencia, el servicio reporta el resultado:

Resultado	Evento publicado	Cola destino
✅ Éxito	<code>ProcesoCreatedEvent</code>	<code>proceso.created</code>
❌ Error	<code>ProcesoCreationFailedEvent</code>	<code>proceso.creation.failed</code>

Paso 5 — Orquestación: Creación del Comité

El orquestador consume `proceso.created`:

1. Actualiza estado SAGA → `PROCESO CREADO`.
 2. Recupera datos del comité desde su BD de control.
 3. Publica → `proceso.comite.create.queue`.
-

Paso 6 — Persistencia Híbrida del Comité

`ms-proceso-comite-service` consume `proceso.comite.create.queue`:

1. Registra comisión en MySQL (`tb_proceso_comite`).
2. **Simultáneamente** almacena documento en MongoDB (`ProcesoComiteDocument`).

Paso 7 — Confirmación o Fallo del Comité

Resultado	Evento publicado	Cola destino
✓ Éxito	ProcesoComiteCreatedEvent	proceso.comite.created
✗ Error	ProcesoComiteCreationFailedEvent	proceso.comite.creation.failed

Paso 8 — Actualización de Estado y Disparo de Notificaciones

El orquestador consume `proceso.comite.created`:

1. Actualiza estado SAGA → `COMITE CREADO`.
 2. Publica → `proceso.comite.notification.queue`.
-

Paso 9 — Envío de Correos Electrónicos

`ms-proceso-comite-notification-service` consume `proceso.comite.notification.queue`:

1. Extrae destinatarios del payload.
 2. Envía correo electrónico a cada integrante del comité.
-

Paso 10 — Confirmación o Fallo de Notificaciones

Resultado	Evento publicado	Cola destino
✓ Éxito	ProcesoComiteNotificatedEvent	proceso.comite.notificated
✗ Error	ProcesoComiteNotificationFailedEvent	proceso.comite.notification.failed

Paso 11 — Cierre de la Transacción Distribuida

El orquestador consume `proceso.comite.notificated`:

1. Actualiza estado SAGA → `COMITE NOTIFICADO`.
2. Actualiza estado SAGA → `COMPLETADO`.

✔ La transacción distribuida se da por concluida exitosamente.

6. Colas RabbitMQ

Cola	Dirección	Productor	Consumidor
proceso.create.queue	→	ms-proceso-orquestador	ms-proceso-service
proceso.created	←	ms-proceso-service	ms-proceso-orquestador
proceso.creation.failed	←	ms-proceso-service	ms-proceso-orquestador
proceso.comite.create.queue	→	ms-proceso-orquestador	ms-proceso-comite-service
proceso.comite.created	←	ms-proceso-comite-service	ms-proceso-orquestador
proceso.comite.creation.failed	←	ms-proceso-comite-service	ms-proceso-orquestador
proceso.comite.notification.queue	→	ms-proceso-orquestador	ms-notification-service
proceso.comite.notificated	←	ms-notification-service	ms-proceso-orquestador
proceso.comite.notification.failed	←	ms-notification-service	ms-proceso-orquestador

Exchange: saga.exchange (Topic Exchange)

7. Modelos de Datos

MySQL — Tabla tb_proceso

sql

```
CREATE TABLE tb_proceso (
  id          VARCHAR(36) PRIMARY KEY,
  nombre      VARCHAR(255) NOT NULL,
  resumen     TEXT,
  titulo      VARCHAR(255),
  cantidad    INT,
  fecha_inicio DATE,
  fecha_fin   DATE,
  organo_id   VARCHAR(36),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

MySQL — BD SAGA (Orquestador)

```
sql

CREATE TABLE saga_state (
  saga_id      VARCHAR(36) PRIMARY KEY,
  status       VARCHAR(50) NOT NULL,    -- STARTED | PROCESO_CREADO | COMITE_CREADO
  updated_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE saga_payload (
  id           BIGINT AUTO_INCREMENT PRIMARY KEY,
  saga_id      VARCHAR(36) NOT NULL,
  payload      JSON NOT NULL,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (saga_id) REFERENCES saga_state(saga_id)
);
```

MySQL — Tabla `tb_proceso_comite`

```
sql

CREATE TABLE tb_proceso_comite (
  id          VARCHAR(36) PRIMARY KEY,
  proceso_id  VARCHAR(36) NOT NULL,
  organo_id   VARCHAR(36),
  nombre      VARCHAR(255),
  tipo        DECIMAL(3,2),
  email       VARCHAR(255),
  FOREIGN KEY (proceso_id) REFERENCES tb_proceso(id)
);
```



```
json
{
  "_id": "ObjectId",
  "sagaId": "411d094d-1180-438e-93c3-5529b1dd4bbe",
  "procesoId": "uuid-proceso",
  "nombre": "PROCESO CAS N° 999-2026-PCM-OGRH",
  "comite": [
    {
      "organoId": "79e69473-34be-4da2-85e2-909291691da4",
      "nombre": "PERCY GIANCARLO GUILLEN ALCANTARA",
      "tipo": 3.00,
      "email": "cyper2@hotmail.com"
    },
    {
      "organoId": "62592a6e-a528-4925-906b-0b432cca4034",
      "nombre": "JUAN PEREZ OLIVA",
      "tipo": 1.00,
      "email": "pguillena@gmail.com"
    }
  ],
  "createdAt": "2026-03-04T00:00:00Z"
}
```

8. Mecanismo de Compensación

Si cualquier paso de la SAGA falla, el orquestador ejecuta **transacciones compensatorias** para revertir los cambios realizados hasta ese punto:

Fallo en Paso 3 (MySQL proceso)

- ↳ Cola: proceso.creation.failed
 - ↳ Orquestador: actualiza estado → FAILED
 - ↳ No hay pasos previos que revertir

Fallo en Paso 6 (MySQL/MongoDB comité)

- ↳ Cola: proceso.comite.creation.failed
 - ↳ Orquestador: actualiza estado → FAILED
 - ↳ Compensación: eliminar registro de tb_proceso
 - ↳ Publica → proceso.compensate.queue

Fallo en Paso 9 (Notificaciones)

- ↳ Cola: proceso.comite.notification.failed
 - ↳ Orquestador: actualiza estado → FAILED
 - ↳ Compensación: eliminar comité (MySQL + MongoDB)
 - + eliminar proceso (MySQL)

9. Request / Response de Ejemplo

Request — POST `/api/saga/proceso`

json

```

{
  "nombre": "PROCESO CAS N° 999-2026-PCM-OGRH",
  "resumen": "CONVOCATORIA PARA LA CONTRATACIÓN ADMINISTRATIVA DE SERVICIOS POR SUPL",
  "titulo": "2DA. CONVOCATORIA",
  "cantidad": 21,
  "fechaInicio": "2026-03-04",
  "fechaFin": "2026-03-04",
  "organoId": "79e69473-34be-4da2-85e2-909291691da4",
  "listaProcesoComite": [
    {
      "organoId": "79e69473-34be-4da2-85e2-909291691da4",
      "nombre": "PERCY GIANCARLO GUILLEN ALCANTARA",
      "tipo": 3.00,
      "email": "cyper2@hotmail.com"
    },
    {
      "organoId": "62592a6e-a528-4925-906b-0b432cca4034",
      "nombre": "JUAN PEREZ OLIVA",
      "tipo": 1.00,
      "email": "pguillena@gmail.com"
    }
  ]
}

```

Response — 200 OK

```

json
{
  "sagaId": "411d094d-1180-438e-93c3-5529b1dd4bbe",
  "message": "Saga started successfully"
}

```

10. Estados de la SAGA

```

STARTED
|
▼ (proceso.created)
PROCESO_CREADO
|
▼ (proceso.comite.created)
COMITE_CREADO
|

```

▼ (proceso.comite.notificated)

COMITE_NOTIFICADO

|

▼

COMPLETADO 

En cualquier punto:

↳ FAILED  (si ocurre un error en cualquier cola *.failed)

11. Consideraciones Técnicas

Idempotencia

Cada microservicio debe garantizar idempotencia en sus operaciones de escritura. Si un mensaje se re-encola por error, no debe generar duplicados en la base de datos.

Dead Letter Queue (DLQ)

Se recomienda configurar una cola `saga.dlq` en RabbitMQ para capturar mensajes que no pudieron procesarse después de N reintentos.

Trazabilidad

El `sagaId` debe propagarse como header en todos los mensajes RabbitMQ y como campo en todos los logs (MDC — Mapped Diagnostic Context) para facilitar el rastreo de una transacción completa.

Persistencia en MongoDB

MongoDB actúa como el **modelo de lectura** (CQRS Read Side) para las consultas de comités. Permite consultas rápidas sin afectar la BD transaccional MySQL.

Configuración RabbitMQ recomendada

yaml

```
rabbitmq:  
  host: localhost  
  port: 5672  
  username: guest  
  password: guest  
  exchange: saga.exchange  
  exchange-type: topic  
  prefetch-count: 1  
  retry:  
    max-attempts: 3  
    initial-interval: 1000ms  
    multiplier: 2
```